

MODULE 6

MACHINE LEARNING WITH LANGUAGE MODELS AND AGENTIC WORKFLOWS

MLFP — ML Foundations for Professionals — Terrene Open Academy

"The best LLM application is not a chatbot. It is an engineer that reasons, acts, and governs itself."

WHAT YOU WILL LEARN

BY THE END OF MODULE 6

- **Prompt engineer** LLMs with 5+ techniques for structured output
- **Fine-tune** models with LoRA and adapter layers from scratch
- **Align** models with DPO and GRPO preference optimization
- **Build RAG systems** with dense, sparse, and hybrid retrieval
- **Create AI agents** with ReAct reasoning and tool use
- **Orchestrate multi-agent systems** with MCP servers
- **Govern AI** with PACT access controls and operating envelopes
- **Deploy production platforms** via Nexus (API + CLI + MCP)

THREE DEPTH LAYERS

FOUNDATIONS — Everyone follows. Plain language, analogies.

THEORY — Mathematical depth. Derivations, code.

ADVANCED — Paper references, edge cases for experts.

A product manager and a research engineer sit in the same classroom. Both leave having learned something new.

YOUR JOURNEY: 8 LESSONS

#	Lesson	You Will Be Able To...
6.1	LLM Fundamentals & Prompt Engineering	Use 5+ prompting techniques for structured output
6.2	LoRA, Adapters & Fine-tuning Landscape	Implement LoRA and adapters from scratch
6.3	Preference Alignment — DPO & GRPO	Train models to follow human preferences
6.4	RAG Systems	Build retrieval-augmented generation pipelines
6.5	AI Agents — ReAct & Tool Use	Build agents that reason and act with cost budgets
6.6	Multi-Agent Orchestration & MCP	Coordinate specialist agents and expose tools via MCP
6.7	AI Governance Engineering	Implement PACT access controls and operating envelopes
6.8	Capstone — Full Production Platform	Deploy a governed AI system via Nexus

KAILASH ENGINES YOU WILL MEET

KAIZEN

`Delegate` — LLM calls with streaming, cost tracking, structured output.

`BaseAgent` — Custom agents with tool use and reasoning.

`Signature` — Type-safe input/output schemas.

KAILASH-ALIGN

`AlignmentPipeline` — DPO, GRPO, and LoRA fine-tuning.

`AdapterRegistry` — Store, version, and swap adapters.

PACT

`GovernanceEngine` — D/T/R access controls, five-dimension constraint envelopes.

`GovernedSupervisor` (from `kaizen_agents`) — Agents with built-in pact governance.

NEXUS

`Nexus` — Deploy one codebase as API + CLI + MCP simultaneously. JWT auth, middleware, rate limiting.

WHERE WE ARE

Module	You Learned	Key Engines
M1	Python, Polars, data pipelines	DataExplorer, ModelVisualizer
M2	Statistics, probability, hypothesis testing	ExperimentTracker
M3	Supervised ML, evaluation, pipelines	TrainingPipeline, HyperparameterSearch
M4	Unsupervised ML, NLP, anomaly detection	AutoMLEngine, DriftMonitor
M5	Deep learning, CNNs, RNNs, transformers	OnnxBridge, InferenceServer

Today: We take the transformer architecture from M5 and give it the ability to reason, act, learn from feedback, and govern itself. Everything you built before becomes a tool an agent can wield.

THE LLM OBSERVATORY

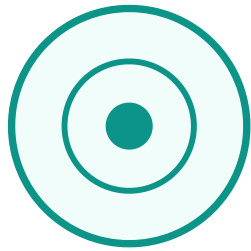
SIX LENSES

What separates amateur prompt-play from production LLM engineering

"M5 gave you a Doctor's Bag for deep networks. M6 needs an Observatory, because a generation is not a loss value."

Turn to your neighbour (2 min): name a time you deployed an LLM and something weird happened — and how you figured out what. *(Self-study: jot it in your notebook.)*

THE SIX LENSES



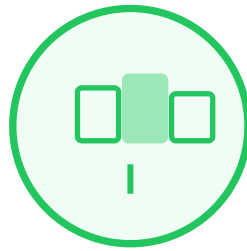
Output

coherent?
faithful?



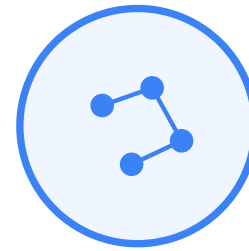
Attention

what does it
attend to?



Retrieval

right chunks
fetched?



Agent Trace

what did the
agent do?



Alignment

aligned with
our intent?



Governance

within
policy?

DIAGNOSE — what is happening?

EVALUATE — training signal?

GOVERN

Each lens answers one question.

First four diagnose the running system. Fifth evaluates the training signal.
Sixth is the organisational contract — did the system stay inside its envelope?

LENS 1: OUTPUT — THE STETHOSCOPE, EVOLVED

ASKS: IS THE GENERATION COHERENT, FACTUAL, ON-TASK?

- **Bounded tasks:** perplexity, ROUGE, BLEU, BERTScore
- **Open-ended:** LLM-as-judge (bias-mitigated — swap positions, normalise length)
- **Hallucination:** self-consistency (sample n, measure agreement) + retrieval grounding
- **Refusal calibration:** refuse harmful prompts, do NOT over-refuse benign ones

Why this lens first? 80% of production LLM failures surface here. If Output is clean, you rarely need to open the Attention lens.

Faithful

prompt: "refund policy?"
answer cites retrieved
policy verbatim

→ ship

Hallucinated

prompt: "refund policy?"
answer invents a
30-day window

→ raise faithfulness

Metrics to run

perplexity	bounded tasks
faithfulness	RAG grounding
LLM-as-judge	open-ended
refusal rate / over-refusal	safety calibration

LENS 1: OUTPUT — CODE

Routes through DeepEval + a Kaizen Delegate judge. No raw `openai.chat.completions.create`.

```
from shared.mfp86.diagnostics import LLMObservatory

obs = LLMObservatory() # lazy init, no network call here

# — Faithfulness against retrieved context —
df = obs.output.faithfulness(
    response="Refunds within 14 days, no exceptions.",
    context=["Refund policy: 30-day window for unused items."],
)
# df columns: claim, supported, support_source, grounding_score

# — Bias-mitigated LLM-as-judge (positions swapped, lengths normalised)
verdict = obs.output.llm_as_judge(
    prompt="Summarise the Q3 earnings call in 3 bullets.",
    response=draft_answer,
    criteria="coherence, factual_accuracy, completeness",
)
# verdict.score in [0, 1], verdict.rationale is a short string

# — Hallucination via self-consistency —
result = obs.output.self_consistency(
    prompt="What is the SOX listing rule for board independence?",
    n=5,
)
# result.agreement_rate, result.flagged_as_hallucination

# — Refusal calibration
obs.output.refusal_rate(harmful_prompts) # should be high
obs.output.over_refusal(benign_prompts)  # should be near zero

obs.output.plot_output_dashboard().show() # single figure, all 4 metrics
```

Budget note: each judge call costs money. The observatory caps spend at **~\$2 per student per module run** via small eval sets and result caching.

LENS 2: ATTENTION — THE X-RAY

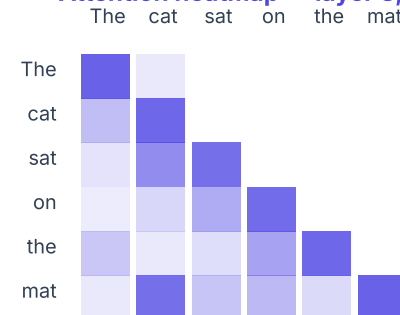
ASKS: WHAT DOES THE MODEL ATTEND TO, INTERNALLY?

- **Attention heatmaps** — token-to-token weights per head
- **Logit lens** — read the model's prediction at every layer (nostalgebraist / Neel Nanda)
- **Sparse autoencoders** — load Gemma Scope features, read what each latent fires on
- **Circuits** — the minimal subgraph that produces a token (activation patching)

Mech interp in 2026: we read *features*, not neurons. The course default for SAE work is **Gemma-2-2B** — Gemma Scope is the best-covered pre-trained SAE suite available.

API-only models (GPT, Claude, Gemini) cannot be X-rayed. The lens reports "not applicable" and falls back to Output + Agent Trace.

Attention heatmap — layer 8, head 3



Read: "mat" attends strongly to "cat" and "sat"
 → this head tracks subject-verb binding

Complement with: `logit_lens` (what layer predicts?)
 and `sae_features` (what concept fires?)

LENS 2: ATTENTION — CODE

Wraps `transformer_lens` + `sae_lens`. Open-weight models only; API models report `not_applicable`.

```
from shared.nlp06.diagnostics import LLMObservatory

obs = LLMObservatory()

# — Attention heatmap for one head —————
fig = obs.attention.attention_heatmap(
    prompt="The cat sat on the mat.",
    model="google/gemma-2-2b",
    layer=8,
    head=3,
)
fig.show()

# — Logit lens: what does each layer predict? —————
df = obs.attention.logit_lens(
    prompt="The Eiffel Tower is located in the city of",
    model="google/gemma-2-2b",
)
# df columns: layer, top_token, top_prob — crystallisation point surfaces

# — Sparse autoencoder features (Gemma Scope) —————
features = obs.attention.sae.features(
    prompt="I felt genuinely happy when the result came in.",
    model="google/gemma-2-2b",
    layer=12,
    top_k=10,
)
# features: [(feature_id, activation, human_readable_label), ...]

# — Circuit trace: minimal subgraph for a target token —————
circuit = obs.attention.circuit_trace(
    prompt="Paris is the capital of",
    target_token=" France",
    model="google/gemma-2-2b",
)
# circuit.nodes: {(layer, component): attribution_score}
```

Non-goal: we do NOT train SAEs in this course. We READ pre-trained ones. SAE training is a research topic; SAE reading is an M6 skill.

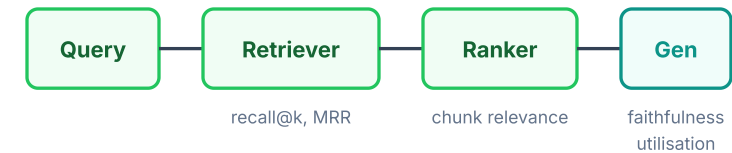
LENS 3: RETRIEVAL — THE RAG DIAGNOSTIC

ASKS: DID WE FETCH THE RIGHT CONTEXT, AND DID WE USE IT?

- **recall@k / precision@k / MRR** — classical IR metrics
- **chunk relevance** — per-chunk LLM-as-judge score
- **context utilisation** — did the generator actually USE the retrieved chunks?
- **retriever bake-off** — BM25 vs dense vs hybrid vs HyDE, same queries

The hidden failure: retrieval can be perfect AND the answer still wrong, because the generator ignored the chunks. You need BOTH recall AND context-utilisation.

RAG pipeline — metrics per stage



Retriever leaderboard — 50 SG policy queries

retriever	recall@5	faithfulness
BM25	0.71	0.88
dense (bge-m3)	0.78	0.81
hybrid	0.83	0.73
HyDE	0.80	0.84

hybrid wins recall, loses faithfulness — more chunks, more ignored conte

LENS 3: RETRIEVAL — CODE

Backs onto `ragas` + `trulens-eval`. Works for any retriever that exposes `retrieve(query, k)`.

```
from shared.nlpdb.diagnostics import LLMObservatory

obs = LLMObservatory()

# — Standard IR metrics —
df = obs.retrieval_recall_at_k(
    queries=eval_queries,
    relevant_ids=gold_chunk_ids,
    retriever=my_retriever,
    k=5,
)

# — Retriever bake-off on identical queries —
leaderboard = obs.retrieval_compare_retrievers(
    retrievers={"bm25": bm25, "dense": dense, "hybrid": hybrid, "hyde": hyde},
    queries=eval_queries,
    relevant_ids=gold_chunk_ids,
    k=5,
)
# Leaderboard: Polars DataFrame with recall@k, precision@k, faithfulness, latency_ms

# — Did the generator actually USE the context? —
util = obs.retrieval_context_utilisation(
    response=generated_answer,
    context=retrieved_chunks,
)
# util.grounded_claims / util.total_claims, util.unused_chunks

# — Chunk-size sweep: find the empirical sweet spot —
sweep = obs.retrieval_chunk_size_sweep(
    documents=policy_corpus,
    sizes=[256, 512, 1024, 2048],
    queries=eval_queries,
)

obs.retrieval_plot_retrieval_dashboard().show() # recall * leaderboard * sweep
```

First use: Exercise 6.4 — Singapore policy RAG leaderboard across four retrievers.

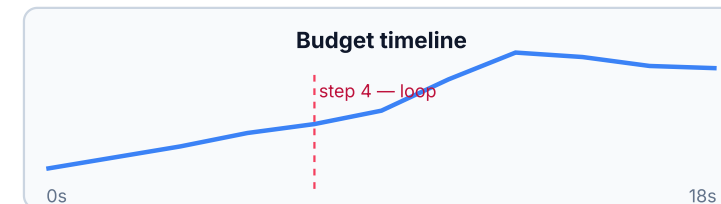
LENS 4: AGENT TRACE — WHAT DID THE AGENT ACTUALLY DO?

- **ReAct capture:** structured Thought / Action / Observation / Done events as JSONL
- **Tool usage:** per-tool call count, success rate, p50/p99 latency, cost share
- **Loop detection:** repeated (thought, action) pairs flagged as cycles
- **Budget timeline:** cumulative cost vs wall clock — where did the money go?
- **Context utilisation:** tokens consumed vs model window over time

Backend: Langfuse self-hosted for production (OTEL-native, Foundation-independence compliant). Colab falls back to a **Plotly-based viewer** because Docker is unavailable.

Annotated ReAct trace — task: "summarise Q3 filings"

1. Thought: list Q3 filings	\$0.002
2. Action: search("Q3 filings 2025")	\$0.004
3. Observation: 12 results	842ms
4. Thought: try "Q3 earnings" — narrower?	loop start
5-14. Action/Obs: re-search 10x	\$0.14 burned



LENS 4: AGENT TRACE — CODE

Wraps Kaizen agent event streams into OpenTelemetry-compatible JSONL.

```
from shared.nlfp06.diagnostics import LLMObservatory
from kaizen_agents import ReActAgent

obs = LLMObservatory()
agent = ReActAgent(tools=[search, calculator, summarise])

# --- Capture a run with full trace ---
with obs.agent.capture(agent, run_id="q3_summary_v1") as trace:
    result = agent.run("Summarise Q3 filings for SIX listed banks.")

# trace is a JSONL file under runs/<agent>/<timestamp>.jsonl

# --- Per-tool cost / latency / success breakdown ---
usage = obs.agent.tool_usage.summary(trace)
# Polars DataFrame: tool, calls, success_rate, p50_ms, p99_ms, cost_usd_total

# --- Loop detection ---
loops = obs.agent.loop_detection(trace)
# Loops: [{"start_step": 4, "length": 10, "signature": "search,observe"}]

# --- Budget timeline plot ---
obs.agent.budget_timeline(trace).show()

# --- Counterfactual replay: what if step 3 returned differently?
replay = obs.agent.replay(
    trace,
    step=3,
    modified_output="0 results",
)
# replay.diverged_at_step, replay.new_total_cost, replay.new_outcome

# --- Optional: export to Langfuse for the UI ---
obs.agent.export_to_langfuse(trace, project="nlfp06-dev")
```

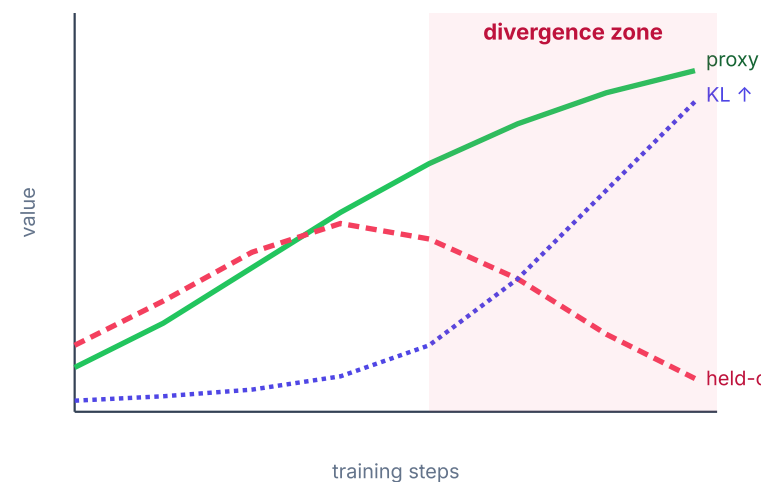
LENS 5: ALIGNMENT — IS THE TRAINING SIGNAL HONEST?

ASKS: HAS THE POLICY DRIFTED? DID IT REWARD-HACK?

- **KL from reference** — per-prompt divergence from the frozen base policy
- **Reward margin** — DPO chosen-minus-rejected log-prob gap distribution
- **Bias-mitigated win rate** — position swap + length normalisation against base
- **Benchmark delta** — MMLU / HellaSwag / TruthfulQA before-vs-after, flags regressions
- **Reward hacking scan** — high proxy score + failed held-out rubric = red flag

GRPO extension: group-relative reward variance and group-collapse detection — GRPO's failure shape is unlike PPO's.

Training curve — reward hacking signature

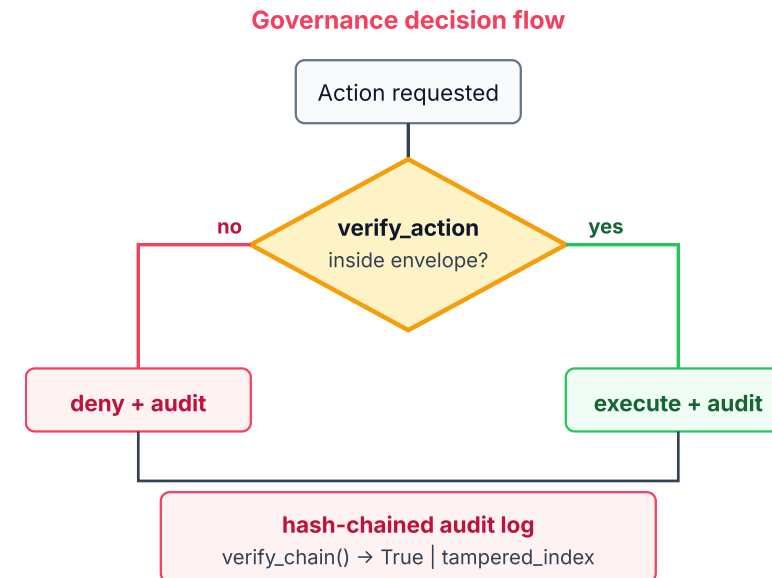


LENS 6: GOVERNANCE — PACT AUDIT

ASKS: IS THE SYSTEM INSIDE ITS ENVELOPE, IS THE AUDIT CHAIN INTACT?

- **Envelope snapshot** — Financial, Operational, Temporal, Data Access, Communication per role
- **Budget consumption** — spend, tokens, tool calls against each envelope dimension
- **Audit chain verification** — hash-linked log, tamper-evident, returns first bad index on failure
- **D/T/R addressing** — who decided, who trusted, who responded (PACT grammar)
- **Negative verdict drill** — assert the DENY set stays denied; positive regression test

Native to Kailash: this lens has zero external dependencies. PACT is the Foundation's own governance framework — the lens inspects GovernanceEngine, not a third-party policy engine.



MODERN LLM OBSERVABILITY STACK (2026)

Layer	Tool	Purpose
Output eval	DeepEval, RAGAS	Faithfulness, hallucination, LLM-as-judge (bias-mitigated)
Interpretability	TransformerLens, nnterp, Gemma Scope SAEs	Attention heatmaps, logit lens, feature reading (read, not train)
Retrieval eval	RAGAS, TruLens	recall@k, chunk relevance, context utilisation, retriever bake-off
Agent trace	Langfuse (self-hosted), LangSmith, Phoenix	TAOD capture, OTEL spans, replay; Plotly viewer fallback for Colab
Alignment	TRL (DPO/GRPO), lm-evaluation-harness	KL, reward margin, bench deltas, reward-hacking scans
Governance	PACT (Foundation-native)	Envelope, audit chain, D/T/R addressing, negative verdict drill

Why this stack? Every tool is open-source or Foundation-native. Langfuse self-hosted over LangSmith; DeepEval over proprietary evals. Independence is a design constraint, not a suggestion.

HOW THE OBSERVATORY THREADS THROUGH M6

Lesson	Topic	Primary lens	Secondary lens
6.1	Prompting & structured output	Output	Attention (preview only — API models)
6.2	LoRA & adapters	Alignment	Attention + Output (attribution)
6.3	DPO & GRPO	Alignment	Output (bias-mitigated judging)
6.4	RAG systems	Retrieval	Output (faithfulness)
6.5	ReAct agents & tool use	Agent Trace	Output (answer quality)
6.6	Multi-agent + MCP	Agent Trace	Governance (envelope breaches)
6.7	AI governance engineering	Governance	Agent Trace (deny patterns)
6.8	Capstone — production platform	ALL SIX — full observatory dashboard	

Every lesson's first exercise begins with: "Run the observatory. Report the relevant lens readings. Then build."

YOUR OBSERVATORY WORKFLOW

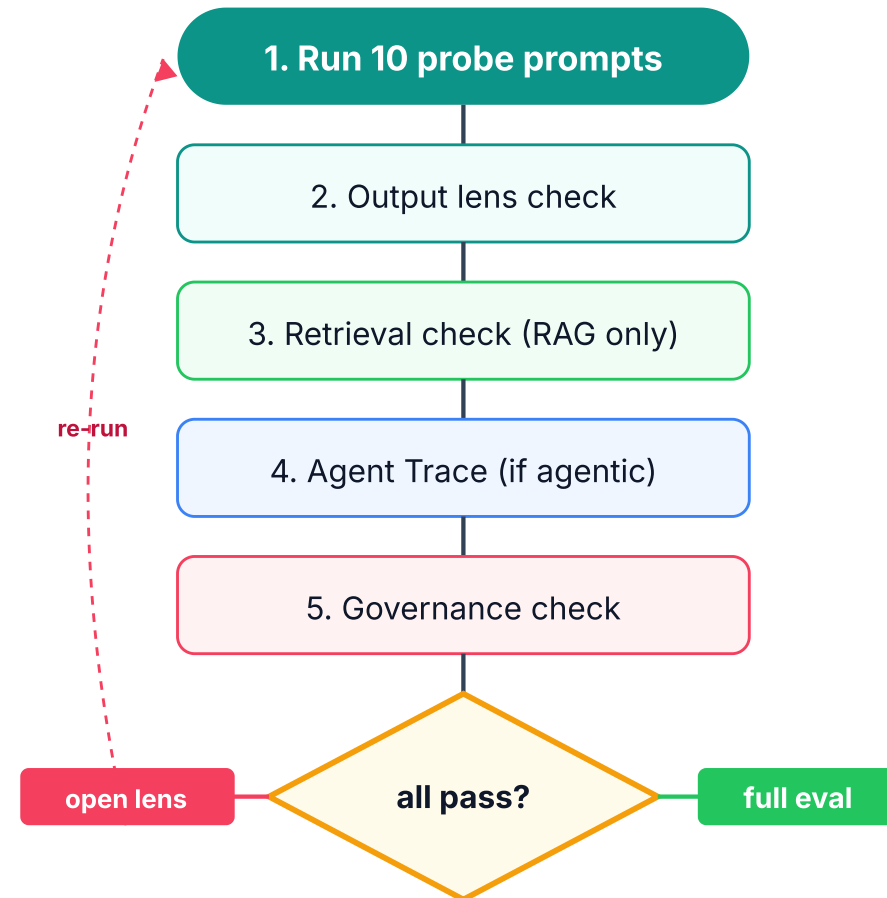
THE PROTOCOL — EVERY LLM SYSTEM, EVERY RUN

Five checks, then a decision.

1. **Run 10 probe prompts** (smoke test, not full eval)
2. **Output lens** — faithfulness & judge scores on the 10
3. **Retrieval lens** (if RAG) — recall@5 + context utilisation
4. **Agent Trace lens** (if agentic) — loop detection + cost budget
5. **Governance lens** (always, if PACT wired) — verify_chain + envelope

Decision: all pass → full eval. Any fails → open the relevant lens's deep-dive methods; change ONE thing; re-run.

Cardinal rule: one change per iteration. Change prompt + retriever + judge model simultaneously and you cannot attribute the fix.



LESSON 6.1: LLM FUNDAMENTALS, PROMPT ENGINEERING & STRUCTURED OUTPUT

LEARNING OBJECTIVES

- Explain how LLMs are pre-trained and aligned
- Apply 5+ prompt engineering techniques effectively
- Use Kaizen `Delegator` for structured LLM output with cost tracking
- Understand basic inference optimisation concepts

Prerequisites: M5 complete (deep learning, transformer architecture).

HOW LLMS LEARN: PRE-TRAINING

NEXT TOKEN PREDICTION (GPT-STYLE)

- Given tokens $t_1, t_2, \dots, t_n$, predict t_{n+1}
- Trained on trillions of tokens from the internet
- The model learns grammar, facts, reasoning — all from prediction
- **Autoregressive**: each token depends on all previous tokens

MASKED LANGUAGE MODELLING (BERT-STYLE)

- Mask 15% of tokens, predict the masked ones
- Bidirectional: sees both left and right context
- Better for classification; worse for generation

$$\mathcal{L} = - \sum_{i=1}^N \log P(t_i | t_1, \dots, t_{i-1}; \theta)$$

Cross-entropy loss: maximise the probability of the next correct token

Key insight: Pre-training does not teach the model to follow instructions. It teaches it to complete text. Alignment (RLHF/DPO) is what makes it helpful.

SCALING LAWS: PARAMETERS, DATA, COMPUTE

THE SCALING HYPOTHESIS

- Performance improves predictably with more **parameters, data, and compute**
- Kaplan et al. (2020): power-law relationship between loss and each factor
- Chinchilla (2022): optimal allocation is roughly equal scaling of params and data

$$L(N, D) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

Loss as a function of model size N and dataset size D

NOTABLE MODELS

Model	Params	Key Innovation
GPT-4	~1.8T (MoE)	Mixture of Experts
Claude	Undisclosed	Constitutional AI
Llama 3	8B-405B	Open weights, GQA
Mistral	7B-8×22B	Sliding window attn
Gemma 2	2B-27B	Knowledge distillation
Phi-3	3.8B-14B	High-quality data curation

FROM PREDICTION TO HELPFULNESS: RLHF



WHY RLHF?

- Pre-training optimises *prediction*, not *helpfulness*
- SFT teaches format (instruction following)
- RLHF teaches quality (prefer helpful, harmless, honest)

THE COMPLEXITY PROBLEM

- Requires training a separate reward model
- PPO is notoriously unstable for LLMs
- Two models in memory: policy + reward
- **Lesson 6.3 solution:** DPO eliminates the reward model entirely

PROMPT ENGINEERING: ZERO-SHOT & FEW-SHOT

ZERO-SHOT

Task description only. No examples.

```
prompt = """Classify the sentiment of this review  
as 'positive' or 'negative':  
Review: 'The food was terrible and the service slow.'  
Sentiment:"""
```

Works well for simple, well-defined tasks. Fails on ambiguous or domain-specific problems.

FEW-SHOT

Provide examples to set the pattern.

```
prompt = """Classify sentiment as 'positive' or 'negative':  
Review: 'Amazing experience!' -> positive  
Review: 'Would not recommend.' -> negative  
Review: 'The food was terrible and the service slow.'  
Sentiment:"""
```

Few-shot is in-context learning: the model infers the task from examples without weight updates.

CHAIN-OF-THOUGHT PROMPTING

STANDARD COT (FEW-SHOT)

```
prompt = """Q: Roger has 5 tennis balls. He buys  
2 cans of 3. How many does he have?  
A: Roger starts with 5. 2 cans of 3 = 6.  
5 + 6 = 11. The answer is 11.
```

```
Q: A restaurant had 23 apples. They used 20  
for lunch and bought 6 more. How many?  
A: Let's think step by step."""
```

ZERO-SHOT COT

```
# Just append "Let's think step by step"  
prompt = """Q: A restaurant had 23 apples.  
They used 20 for lunch and bought 6 more.  
How many apples do they have?
```

```
Let's think step by step."""
```

Kojima et al. (2022): Adding "Let's think step by step" improves accuracy on GSM8K from 17.7% to 78.7% (zero-shot CoT vs standard zero-shot).

SELF-CONSISTENCY & STRUCTURED PROMPTING

SELF-CONSISTENCY

- Sample multiple CoT reasoning paths (temperature > 0)
- Majority vote on final answers
- More robust than single CoT — reduces variance

```
# Sample 5 CoT paths, take majority answer
answers = []
for _ in range(5):
    response = delegate.generate(
        prompt, temperature=0.7
    )
    answers.append(extract_answer(response))
final = majority_vote(answers)
```

STRUCTURED OUTPUT

- Specify output format: JSON, tables, schemas
- Critical for production: free-form text is unparseable
- Kaizen `Signature` enforces this at the framework level

```
prompt = """Analyse this text and return JSON:
{
  "sentiment": "positive|negative|neutral",
  "confidence": 0.0-1.0,
  "key_phrases": ["phrase1", "phrase2"]
}

Text: "The quarterly results exceeded..."
"""
```

PROMPT ENGINEERING: THE COMPLETE TOOLKIT

Technique	When to Use	Cost	Accuracy Gain
Zero-shot	Simple, well-defined tasks	Lowest	Baseline
Few-shot	Pattern-matching, classification	Low-Medium	+10-30%
CoT (few-shot)	Multi-step reasoning	Medium	+20-60%
Zero-shot CoT	Reasoning without examples	Low	+15-50%
Self-consistency	High-stakes decisions	High (Nx)	+5-15% over CoT
Structured output	Production pipelines	Low	Parseability, not accuracy

Engineering rule: Start with zero-shot. Add few-shot if accuracy is insufficient. Add CoT if reasoning is needed. Add self-consistency only if the cost is justified by the stakes.

KAIZEN: DELEGATE & SIGNATURE

FROM RAW PROMPTS TO TYPE-SAFE OUTPUT

```
import os
from dataclasses import dataclass
from kaizen import Signature, InputField, OutputField
from kaizen.core.base_agent import BaseAgent

class SentimentSignature(Signature):
    """Classify sentiment of a product review."""
    review: str = InputField(description="Product review text")
    sentiment: str = OutputField(description="positive, negative, or neutral")
    confidence: float = OutputField(description="Confidence score 0.0 to 1.0")

@dataclass
class SentimentConfig:
    llm_provider: str = os.environ.get("LLM_PROVIDER", "openai")
    model: str = os.environ.get("LLM_MODEL", "gpt-4o-mini")
    temperature: float = 0.2
    budget_limit_usd: float = 1.0

class SentimentAgent(BaseAgent):
    def __init__(self, config: SentimentConfig):
        super().__init__(config.config, signature=SentimentSignature())
        agent = SentimentAgent(SentimentConfig())
        result = await agent.run_async(
            review="The battery life is incredible but the screen is dim."
        )
        print(result["sentiment"])  # "mixed" or "neutral"
        print(result["confidence"])  # 0.72
```

Why Signature? Free-form text output is brittle. Signature enforces a schema: the LLM must return typed fields. If it does not, Kaizen retries with corrective prompting.

INFERENCE OPTIMISATION (BRIEF)

KV-CACHE

Cache key/value tensors for previously processed tokens. Avoids redundant computation during autoregressive generation. Reduces latency by 2-5x.

SPECULATIVE DECODING

Use a small draft model to propose tokens, then verify in batch with the large model. Acceptance rate typically 70-90%. Faster without quality loss.

CONTINUOUS BATCHING

New requests join the batch as slots free up, rather than waiting for the entire batch to complete. Maximises GPU utilisation for serving.

Production relevance: You will not implement these yourself. But understanding them explains why some APIs are faster than others and why batch pricing exists. Lesson 6.8 covers vLLM for production serving.

EXERCISE 6.1: PROMPT ENGINEERING SHOWDOWN

TASK

- Build a classification system using Kaizen
Delegate
- Compare zero-shot vs few-shot vs CoT on the same dataset
- Measure cost and accuracy for each approach
- Use Signature for all outputs

```
# Skeleton
import os
from kaizen import Signature, InputField, OutputField
from kaizen.agents import Delegate

class ClassificationResult(Signature):
    """Classify news article topic"""
    article: str = InputField()
    topic: str = OutputField()
    reasoning: str = OutputField()

delegate = Delegate(
    model=os.environ.get("LLM_MODEL", "gpt-4o-mini"),
    budget_usd=0.50,
    signature=ClassificationResult,
)

# TODO: Implement zero_shot_classify()
# TODO: Implement few_shot_classify()
# TODO: Implement cot_classify()
# TODO: Compare accuracy & cost
```

ASSESSMENT CRITERIA

- All three prompting techniques demonstrated
- Accuracy comparison shows when each technique helps
- Cost tracking working — total spend reported

LESSON 6.2: LLM FINE-TUNING — LORA, ADAPTERS & THE TECHNIQUE LANDSCAPE

LEARNING OBJECTIVES

- Implement LoRA from scratch (understand the mathematics)
- Implement adapter layers from scratch
- Compare LoRA vs adapters across 4 dimensions
- Survey the full fine-tuning landscape and select the right technique
- Explain model merging techniques and when to use them

Prerequisites: 6.1 (LLM fundamentals), M5 Lesson 4 (transformer architecture).

LORA: LOW-RANK ADAPTATION

CORE IDEA

- Pre-trained weights W_0 are frozen
- Weight update decomposed into two small matrices
- Instead of updating all $d \times d$ parameters, learn $d \times r$ and $r \times d$
- Rank $r \ll d$ — typically 4, 8, or 16

M4 Connection: LoRA IS SVD applied to weight updates. The low-rank matrices B and A approximate the full update ΔW just like truncated SVD approximates a matrix.

$$W = W_0 + \Delta W = W_0 + BA$$

$$W_0 \in \mathbb{R}^{d \times d} \text{ (frozen)} \mid B \in \mathbb{R}^{d \times r} \mid A \in \mathbb{R}^{r \times d}$$

$$h = W_0x + BAx$$

Forward pass: original output + low-rank adaptation

Parameter savings: For $d = 4096, r = 8$: full update = 16.7M params. LoRA = 65K params. **256x reduction.**

LORA: FROM-SCRATCH IMPLEMENTATION

```
import torch
import torch.nn as nn

class LoRALayer(nn.Module):
    """Low-Rank Adaptation Layer."""
    def __init__(self, in_features: int, out_features: int, rank: int = 8):
        super().__init__()
        self.rank = rank
        # Frozen original weights
        self.linear = nn.Linear(in_features, out_features, bias=False)
        self.linear.weight.requires_grad = False

        # Low-rank matrices
        self.A = nn.Parameter(torch.randn(rank, in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(out_features, rank))
        # A is initialised with small random values; B with zeros
        # So BA = 0 at init: adaptation starts from the original model

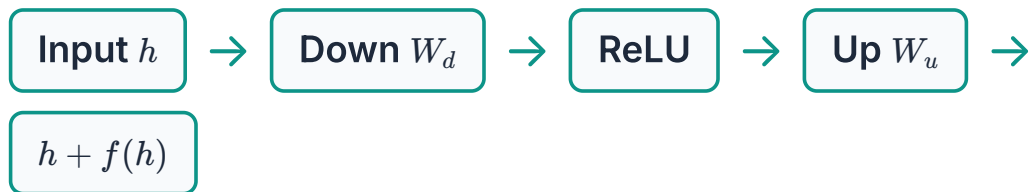
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        original = self.linear(x)          # H_0 @ x
        adaptation = (x @ self.A.T) @ self.B.T # x @ A^T @ B^T = (BA)x
        return original + adaptation
```

Why $B=0$? At initialisation, $BA = 0$, so the model starts as the original pre-trained model. Training gradually learns the adaptation. This is why LoRA is safe: you cannot make the model worse at the start.

ADAPTER LAYERS: BOTTLENECK MODULES

ARCHITECTURE

- Insert small bottleneck modules between transformer layers
- Down-project → Non-linearity → Up-project
- Residual connection: if adapter output is zero, original behaviour preserved
- Only adapter weights are trained; transformer weights frozen



```
class AdapterLayer(nn.Module):
    def __init__(self, hidden_size: int,
                  bottleneck: int = 64):
        super().__init__()
        self.down = nn.Linear(
            hidden_size, bottleneck)
        self.activation = nn.ReLU()
        self.up = nn.Linear(
            bottleneck, hidden_size)
        # Init up-project near zero
        nn.init.zeros_(self.up.weight)
        nn.init.zeros_(self.up.bias)

    def forward(self, x: torch.Tensor):
        residual = x
        x = self.down(x)
        x = self.activation(x)
        x = self.up(x)
        return residual + x # Skip conn
```

LORA VS ADAPTERS: HEAD-TO-HEAD

Dimension	LoRA	Adapters
Parameter update	Low-rank additive ($W_0 + BA$)	Bottleneck modules (down/up projection)
Parameter efficiency	Higher (rank controls size)	Lower (bottleneck size controls)
Inference latency	Zero overhead (merge weights)	Adds sequential computation per layer
Modularity	Can merge into base weights	Separate modules, hot-swappable
Flexibility	Applied to any weight matrix	Inserted at fixed positions
Typical use	Production fine-tuning (dominant)	Multi-task serving (swap adapters)

Engineering verdict: LoRA dominates production use because merged weights have zero inference overhead. Adapters remain useful when you need to swap between many tasks without reloading the model.

THE FINE-TUNING LANDSCAPE: 10 TECHNIQUES

Technique	What Changes	Key Idea
LoRA	Low-rank matrices	Additive weight decomposition
Adapters	Bottleneck modules	Sequential down/up projection
Prefix Tuning	K/V prefix vectors	Task-specific attention context
Prompt Tuning	Soft prompt tokens	Learnable input embeddings
Full Fine-tuning	All parameters	LR scheduler + gradient clipping + mixed precision
LLRD	All params, variable LR	Lower LR for earlier (general) layers
Progressive Freezing	Top layers first	Unfreeze top-down over epochs
Distillation	Student model	Teacher soft labels + KL divergence
DP-SGD	All params + noise	Gradient noise for privacy guarantees
EWC	All params + Fisher penalty	Prevent catastrophic forgetting

Decision rule: Start with LoRA (fastest, cheapest). Full fine-tuning only if domain is very different from pre-training data. Distillation for deployment on constrained hardware. DP-SGD when privacy is a legal requirement.

MODEL MERGING: COMBINING FINE-TUNED MODELS

TIES

Trim small values, elect sign by majority, merge. Resolves sign conflicts between adapters.

DARE

Drop and rescale: randomly drop weight deltas, rescale survivors. Reduces interference.

SLERP

Spherical linear interpolation between weight vectors. Smooth blending of two models.

TASK ARITHMETIC

Add or subtract task vectors:
 $\tau = W_{\text{ft}} - W_0$. Combine: $W = W_0 + \lambda(\tau_1 + \tau_2)$.

Application: Train separate LoRA adapters for sentiment analysis and summarisation. Merge them with TIES to get a model that does both — without retraining.

QUANTISATION: RUNNING LARGE MODELS ON SMALL HARDWARE

QUANTISATION METHODS

Method	Bits	Approach
GPTQ	4-bit	Post-training, layer-wise
AWQ	4-bit	Activation-aware weights
GGUF	2-8 bit	CPU-optimised format
bitsandbytes	4/8-bit	Dynamic quantisation

QLORA: THE BEST OF BOTH

- Quantise base model to 4-bit (NF4 datatype)
- Add LoRA adapters in full precision (BF16)
- Train LoRA while base stays quantised
- **Result:** Fine-tune a 65B model on a single 48GB GPU

Dettmers et al. (2023): QLoRA matches full 16-bit fine-tuning quality while using 4x less memory.

KAILASH-ALIGN: FINE-TUNING WITH ALIGNMENTPIPELINE

ALIGNMENTPIPELINE — FROM THEORY TO ONE CALL

```
from kailash_align import AlignmentPipeline, AlignmentConfig, AdapterRegistry

# Configure LoRA fine-tuning
config = AlignmentConfig(
    method="lora",
    model_name="meta-llama/Llama-3-8B",
    rank=8,
    alpha=16,
    target_modules=["q_proj", "v_proj"],
    learning_rate=2e-4,
    num_epochs=3,
)

# Train
pipeline = AlignmentPipeline(config)
result = pipeline.train(dataset="imdb_sentiment")

# Store adapter for later use
registry = AdapterRegistry()
registry.register("sentiment-lora-v1", result.adapter)
print(f"Trainable params: {result.trainable_params:,}") # ~262K
```

EXERCISE 6.2: FROM-SCRATCH FINE-TUNING

TASKS

1. Implement `LoRALayer` from scratch
2. Implement `AdapterLayer` from scratch
3. Fine-tune on IMDB sentiment with both approaches
4. Compare: accuracy, parameter count, training time
5. Merge two LoRA adapters with TIES

ASSESSMENT CRITERIA

- Both from-scratch implementations produce correct gradients
- LoRA achieves comparable accuracy to full fine-tuning
- Quantitative comparison table produced
- Merged model retains both capabilities

Common mistake: Forgetting to freeze the original weights. If `W_0.requires_grad = True`, you are doing full fine-tuning, not LoRA.

LESSON 6.3: PREFERENCE ALIGNMENT — DPO & GRPO

LEARNING OBJECTIVES

- Derive DPO from the RLHF objective
- Implement DPO training with preference pairs
- Explain GRPO and when to prefer it over DPO
- Evaluate fine-tuned models using LLM-as-judge and standard benchmarks

Prerequisites: 6.2 (fine-tuning), M5 Lesson 8 (reinforcement learning, PPO).

DPO: DIRECT PREFERENCE OPTIMIZATION

FROM RLHF TO DPO

- RLHF objective: maximise reward while staying close to reference policy
- Rafailov et al. (2023): the optimal policy has a **closed-form solution**
- Rearranging: we can express the reward in terms of the policy directly
- Result: train directly on preference pairs — no reward model needed

Bradley-Terry model: The probability that response y_w is preferred over y_l follows a logistic function of their reward difference.

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

y_w = preferred response | y_l = rejected response | β = deviation penalty

Intuition: Increase probability of y_w relative to reference. Decrease probability of y_l relative to reference. β controls how far the policy can deviate.

DPO: TRAINING LOOP

```
import torch
import torch.nn.functional as F

def dpo_loss(
    policy_chosen_logps: torch.Tensor, # log pi(y_w|x)
    policy_rejected_logps: torch.Tensor, # log pi(y_l|x)
    ref_chosen_logps: torch.Tensor, # log pi_ref(y_w|x)
    ref_rejected_logps: torch.Tensor, # log pi_ref(y_l|x)
    beta: float = 0.1,
) → torch.Tensor:
    """Direct Preference Optimization loss."""
    chosen_rewards = beta * (policy_chosen_logps - ref_chosen_logps)
    rejected_rewards = beta * (policy_rejected_logps - ref_rejected_logps)
    loss = -F.logsigmoid(chosen_rewards - rejected_rewards).mean()
    return loss

# Training loop (simplified)
for batch in preference_data_loader:
    chosen_logps = model.log_prob(batch["chosen"])
    rejected_logps = model.log_prob(batch["rejected"])
    ref_chosen_logps = ref_model.log_prob(batch["chosen"])
    ref_rejected_logps = ref_model.log_prob(batch["rejected"])

    loss = dpo_loss(chosen_logps, rejected_logps,
                    ref_chosen_logps, ref_rejected_logps)
    loss.backward()
    optimizer.step()
```


GRPO: GROUP RELATIVE POLICY OPTIMIZATION

KEY INNOVATION (DEEPSEEK-R1, 2025)

- Sample G completions for each prompt
- Score each completion with a verifier (not a learned reward model)
- Compute advantage relative to group mean
- Update policy with clipped objective (like PPO, but group-normalised)

$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

Advantage of completion i relative to group statistics

DPO VS GRPO

Aspect	DPO	GRPO
Input data	Preference pairs	Single prompts
Reward signal	Implicit (from pairs)	Verifier scores
Best for	Subjective quality	Objective tasks (code, math)
Stability	Very stable	Stable (group norm)

When to use GRPO: Tasks with verifiable answers (code passes tests, math has correct answer). DPO is better for open-ended quality alignment.

LLM-AS-JUDGE EVALUATION

USING ONE LLM TO EVALUATE ANOTHER

```
judge_prompt = """Rate this response on a  
scale of 1-5 for helpfulness.
```

```
User query: {query}  
Response A: {response_a}  
Response B: {response_b}
```

```
Return JSON:  
{  
  "winner": "A" or "B",  
  "reasoning": "...",  
  "score_a": 1-5, "score_b": 1-5  
}
```

KNOWN BIASES

Bias	Mitigation
Position bias	Swap A/B order, average scores
Verbosity bias	Normalise by response length
Self-enhancement	Use different model as judge

Never trust a single judge call. Always: swap positions, run multiple times, aggregate. LLM-as-judge is a statistical estimator, not a deterministic oracle.

EVALUATION BENCHMARKS

Benchmark	Measures	Task Type
MMLU	Multi-task language understanding	57 subjects, multiple choice
HellaSwag	Commonsense reasoning	Sentence completion
HumanEval	Code generation	Python function completion
MT-Bench	Multi-turn conversation quality	LLM-as-judge scoring
GSM8K	Grade-school math reasoning	Word problems

```
# lm-eval-harness: unified evaluation framework
pip install lm-eval
lm_eval --model hf --model_args pretrained=my-model \
  --tasks mmlu,hellaswag,gsm8k --batch_size 8
```

Best practice: Run benchmarks before and after alignment. If MMLU drops more than 2%, your alignment is degrading general capability. This is the *alignment tax*.

KAILASH-ALIGN: DPO TRAINING

ALIGNMENTPIPELINE WITH METHOD="DPO"

```
from kailash_align import AlignmentPipeline, AlignmentConfig

config = AlignmentConfig(
    method="dpo",
    model_name="meta-llama/Llama-3-8B",
    beta=0.1,          # Deviation penalty
    lora_rank=8,       # Use LoRA for efficiency
    learning_rate=5e-5,
    num_epochs=1,
)

pipeline = AlignmentPipeline(config)
result = pipeline.train(
    dataset="preference_pairs", # (prompt, chosen, rejected) triples
    eval_with="llm_judge",     # Auto-evaluate with LLM-as-judge
)

print(f"Win rate vs reference: {result.win_rate:.1%}")
print(f"MMLU change: {result.mmlu_delta:+.1f}%")
```

EXERCISE 6.3: PREFERENCE ALIGNMENT

TASKS

1. Fine-tune a model with DPO on preference data
2. Evaluate using LLM-as-judge (measure position and verbosity bias)
3. Run lm-eval benchmarks before and after alignment
4. Report the alignment tax (capability degradation)

ASSESSMENT CRITERIA

- DPO training converges (loss decreases)
- LLM-as-judge biases measured and mitigated
- Benchmarks show alignment impact quantitatively
- Win rate vs reference model reported

LESSON 6.4: RAG SYSTEMS

LEARNING OBJECTIVES

- Build a complete RAG pipeline from documents to answers
- Compare dense, sparse, and hybrid retrieval approaches
- Evaluate RAG quality using RAGAS metrics
- Implement HyDE for improved retrieval

Prerequisites: 6.1 (LLM fundamentals, prompt engineering), M4 Lesson 6 (NLP, embeddings).

RAG: RETRIEVAL-AUGMENTED GENERATION



WHY RAG?

- LLMs have a **knowledge cutoff** — no real-time information
- LLMs **hallucinate** — confidently wrong answers
- RAG grounds the model in actual documents
- Cheaper than fine-tuning: update documents, not weights

THE CORE PATTERN

1. **Offline:** Chunk documents, embed chunks, store in vector DB
2. **Online:** Embed user query, retrieve similar chunks
3. **Generate:** Pass query + retrieved chunks to LLM

RAG is an open-book exam for the LLM: it can look up the answer instead of relying on memory.

CHUNKING STRATEGIES

Strategy	Method	Pros	Cons
Fixed-size	Split every N tokens	Simple, predictable	Breaks mid-sentence
Sentence	Split on sentence boundaries	Coherent units	Variable size
Paragraph	Split on paragraph breaks	Preserves context	Can be too large
Semantic	Embed sentences, cluster similar	Meaning-preserving	Expensive to compute

OVERLAP

Chunks overlap by 10-20% to preserve context at boundaries. Without overlap, a fact split across two chunks is lost to both.

CHUNK SIZE TRADEOFFS

Small chunks (128-256 tokens): precise retrieval, more noise.

Large chunks (512-1024 tokens): more context, less precise matching.

Sweet spot: 256-512 tokens for most use cases.

RETRIEVAL: DENSE, SPARSE, AND HYBRID

DENSE RETRIEVAL

Embed query and documents into vectors. Find nearest neighbours by cosine similarity. Captures semantic meaning.

```
from sentence_transformers import (
    SentenceTransformer)
model = SentenceTransformer(
    "all-MiniLM-L6-v2")
q_emb = model.encode(query)
# Cosine similarity search
```

SPARSE RETRIEVAL (BM25)

Term frequency + inverse document frequency. Exact keyword matching. Fast, interpretable, no GPU needed.

```
from rank_bm25 import BM25Okapi
bm25 = BM25Okapi(tokenized_docs)
scores = bm25.get_scores(
    tokenized_query)
top_k = scores.argsort()[-5:]
```

HYBRID RETRIEVAL

Combine dense + sparse scores. Reciprocal rank fusion. Best of both worlds: semantic understanding + exact matching.

```
def rrf(dense_ranks, sparse_ranks,
        k=60):
    scores = {}
    for doc, r in dense_ranks:
        scores[doc] = 1/(k+r)
    for doc, r in sparse_ranks:
        scores[doc] = scores.get(
            doc, 0) + 1/(k+r)
    return sorted(scores, ...)
```

RE-RANKING & HYDE

CROSS-ENCODER RE-RANKING

- First-stage retrieval: fast but approximate (bi-encoder)
- Second-stage re-ranking: slow but accurate (cross-encoder)
- Cross-encoder sees query + document together — much better relevance scoring

```
from sentence_transformers import (
    CrossEncoder)
reranker = CrossEncoder(
    "cross-encoder/ms-marco-MiniLM-L-6-v2")
pairs = [(query, doc) for doc in candidates]
scores = reranker.predict(pairs)
top_docs = [candidates[i]
             for i in scores.argsort()[-3:]]
```

HYDE (HYPOTHETICAL DOCUMENT EMBEDDINGS)

- Problem: query and document are in different "language spaces"
- Solution: ask the LLM to generate a hypothetical answer
- Embed the hypothetical answer instead of the query
- The hypothetical answer is in "document space" — better cosine similarity

```
hypothetical = delegate.generate(
    f"Write a passage that answers: {query}")
)
hyde_embedding = model.encode(hypothetical)
# Search with hyde_embedding instead
```

RAGAS: RAG EVALUATION FRAMEWORK

Metric	Measures	How
Faithfulness	Is the answer supported by retrieved context?	LLM checks each claim against context
Answer Relevance	Does the answer address the question?	Generate questions from answer, compare to original
Context Relevance	Are retrieved chunks relevant to the question?	LLM scores each chunk's relevance
Context Recall	Did we retrieve all needed information?	Compare retrieved context to ground truth

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_relevancy

results = evaluate(
    dataset=eval_dataset, # questions + ground truth + retrieved contexts + answers
    metrics=[faithfulness, answer_relevancy, context_relevancy],
)
print(results) # {"faithfulness": 0.87, "answer_relevancy": 0.92, ...}
```

EXERCISE 6.4: BUILD A RAG SYSTEM

TASKS

1. Build RAG system on Singapore policy documents
2. Implement BM25, dense, and hybrid retrieval
3. Evaluate with RAGAS metrics
4. Implement HyDE and measure improvement

ASSESSMENT CRITERIA

- RAG pipeline end-to-end functional
- Three retrieval methods compared quantitatively
- RAGAS metrics computed for each approach
- HyDE measurably improves retrieval quality

Data: Singapore government policy documents (public domain). Real data, real questions, real evaluation.

LESSON 6.5: AI AGENTS — REACT, TOOL USE & FUNCTION CALLING

LEARNING OBJECTIVES

- Build ReAct agents with custom tools
- Implement function calling with structured schemas
- Apply cost budgets to prevent runaway LLM spending
- Design agents using the hiring mental framework

Prerequisites: 6.1 (LLM fundamentals, Kaizen Delegate).

WHAT IS AN AI AGENT?

THE CORE LOOP

Observe → Think → Act → Observe

Repeat until task is complete or budget is exhausted.

An agent is an LLM that can **use tools**. Instead of just generating text, it can search the web, run code, query databases, call APIs.

AGENT VS PIPELINE

Pipeline	Agent
Fixed sequence of steps	Dynamic: decides next step
Deterministic	Non-deterministic
Fast, predictable	Flexible, adaptive
Use when path is known	Use when path varies

Engineering rule: If a pipeline can solve it, do not use an agent. Agents add complexity and cost. Use agents when the problem requires *reasoning about which tools to use*.

REACT: REASONING + ACTING

THE REACT LOOP

Thought: I need to find the average HDB price in 2024.

Action:

```
data_explorer.describe(df.filter(pl.col("year") == 2024))
```

Observation: Mean price: \$523,000. Median: \$480,000. Count: 12,345.

Thought: I have the average. Now I should check the trend over the last 5 years.

Action: `visualizer.line_chart(df, x="year", y="price")`

Observation: Upward trend from \$420K (2020) to \$523K (2024).

Thought: I have enough information to answer the user's question.

WHY EXPLICIT REASONING?

- Without "Thought" steps, agents make random tool calls
- Reasoning before acting improves tool selection accuracy
- The trace is debuggable: you can see *why* the agent did what it did

Yao et al. (2023): ReAct outperforms act-only agents on HotpotQA by 6% and reduces hallucination rate by 40%. The "Thought" step is not overhead; it is the mechanism.

FUNCTION CALLING: STRUCTURED TOOL USE

JSON SCHEMA DEFINITIONS

```
tools = [{  
  "name": "search_database",  
  "description": "Search for records",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "query": {  
        "type": "string",  
        "description": "Search query"  
      },  
      "limit": {  
        "type": "integer",  
        "default": 10  
      }  
    },  
    "required": ["query"]  
  }  
}]
```

TOOL CHOICE PARAMETER

Value	Behaviour
auto	Model decides whether to call a tool
required	Must call at least one tool
specific	Must call a named tool

PARALLEL FUNCTION CALLING

Models can invoke multiple tools simultaneously when the calls are independent. Reduces latency for multi-tool queries.

Function calling is how agents actually *do things*. The LLM generates a structured JSON call; your code executes it and returns the result.

AGENT DESIGN: THE HIRING FRAMEWORK

FOUR QUESTIONS

1. **What is our goal?** Clear, measurable outcome
2. **What is our thought process?** Step-by-step reasoning
3. **What specialist would we hire?** Be sharp: "HR research specialist" not "researcher"
4. **What tools do they need?** Versatile, fault-tolerant, caching

DESIGN CONSIDERATIONS

- **Iterative refinement:** Add a critic agent that reviews output and recommends improvements
- **Human-in-the-loop:** Pause workflow for validation on high-stakes decisions
- **Monitoring:** Real-time tracking of intermediate outputs for debugging
- **Cost budget:** Set a maximum spend per task to prevent runaway API calls

Think like a hiring manager: You would not hire a "general analyst." You would hire a "financial fraud investigator with experience in transaction pattern analysis." The more specific your agent's role, the better it performs.

KAIZEN: BUILDING AGENTS

BASEAGENT — CUSTOM AGENT WITH TOOLS

```
import os
from kaizen_agents.agents.specialized.react import ReActAgent
from kailash_ml import DataExplorer, TrainingPipeline

def explore_data(dataset: str) → dict:
    """Profile a dataset and return summary statistics."""
    return DataExplorer().profile(dataset).to_dict()

def train_model(dataset: str, target: str) → dict:
    """Train a classification model on the given dataset."""
    pipeline = TrainingPipeline(task="classification")
    result = pipeline.fit(dataset, target=target)
    return {"accuracy": result.accuracy, "model_id": result.model_id}

agent = ReActAgent(
    model=os.environ.get("LLM_MODEL", "gpt-4o-mini"),
    tools=[explore_data, train_model],
    max_llm_cost_usd=1.00, # $1 budget cap; agent halts when exceeded
)
result = await agent.run(
    "Analyse the HDB resale dataset and predict price categories."
)
```

COST BUDGETS: PREVENTING RUNAWAY SPENDING

THE PROBLEM

- Agents make LLM calls in a loop
- A confused agent can make hundreds of calls before realising it is stuck
- At \$0.003/call, 500 calls = \$1.50 per request
- Production system with 10K daily users = \$15,000/day

THE SOLUTION: MAX_LLM_COST_USD

```
import os
from h2oai_agents.agents.specialized.react import (
    ReActAgent,
)

# Budget lives on the agent itself - no
# separate LLMCostTracker class in 2.7.
agent = ReActAgent(
    model=os.environ.get("LLM_MODEL", "gpt-4o-mini"),
    tools=[explore_data, train_model],
    max_llm_cost=os.getenv(
        "MAX_LLM_COST_USD", 0.0
    ),
)

# When budget exhausted:
# -> Agent stops gracefully
# -> Returns partial results
# -> Exits a budget-exceeded event
result = await agent.run("Analyze HDB data")
print(result)
```

Non-negotiable in production: Every agent must have a cost budget. No exceptions. An unbounded agent is a financial liability.

EXERCISE 6.5: BUILD A DATA ANALYSIS AGENT

TASKS

1. Build ReAct agent wrapping DataExplorer, TrainingPipeline, and ModelVisualizer as tools
2. Agent autonomously: explores data, selects model, trains, reports
3. Implement cost budget enforcement
4. Debug agent reasoning chain

ASSESSMENT CRITERIA

- Agent reasons through steps (not random tool calls)
- Tools correctly invoked with valid parameters
- Cost budget respected — agent stops gracefully
- Results interpretable and correct

LESSON 6.6: MULTI-AGENT ORCHESTRATION & MCP

LEARNING OBJECTIVES

- Implement supervisor-worker, sequential, and parallel multi-agent patterns
- Build an MCP server that exposes ML tools
- Configure agent memory (short-term, long-term, entity)
- Apply security considerations to agent architectures

Prerequisites: 6.5 (single agents, tool use).

MULTI-AGENT PATTERNS

SUPERVISOR-WORKER

One agent delegates to specialists. Supervisor decides who works on what. Best for: complex tasks with clear sub-problems.

SEQUENTIAL

Output of one agent feeds into the next. Chain of specialists. Best for: pipeline-like tasks (analyse → engineer → model).

PARALLEL

Multiple agents work simultaneously. Results aggregated. Best for: independent sub-tasks (search + compute + visualise).

HANDOFF

Agent transfers to specialist when topic changes. Stateful: context passed along. Best for: multi-domain conversations.

Decision rule: Start with sequential (simplest). Add parallelism if sub-tasks are independent. Use supervisor-worker only when task decomposition is dynamic.

MULTI-AGENT: SUPERVISOR-WORKER IN KAIZEN

KAIZEN MULTI-AGENT ORCHESTRATION

```
import os
from kaizen_agents import Delegate, Pipeline

data_scientist = Delegate(
    model=os.environ.get("LLM_MODEL", "gpt-4o-mini"),
    tools=[explore_data],
    budget_usd=2.00,
)
feature_engineer = Delegate(
    model=os.environ.get("LLM_MODEL", "gpt-4o-mini"),
    tools=[engineer_features],
    budget_usd=2.00,
)

# Supervisor orchestrates specialists via kaizen_agents.Pipeline -
# the router selects the best specialist based on A2A capability cards.
supervisor = Pipeline.router(
    agents=[data_scientist, feature_engineer],
)
result = await supervisor.run(
    objective="Profile the HDB dataset and engineer features",
)
```

AGENT MEMORY

SHORT-TERM MEMORY

Current conversation context. Managed by the LLM's context window. Lost when the session ends. Used for: multi-turn reasoning within one task.

LONG-TERM MEMORY

Persistent knowledge across sessions. Stored in a vector database. Retrieved by semantic similarity. Used for: learning from past interactions.

ENTITY MEMORY

Structured knowledge about specific entities (people, projects, datasets). Key-value store with entity extraction. Used for: maintaining facts about recurring subjects.

Production pattern: Short-term memory for the current task. Long-term memory for domain knowledge. Entity memory for user-specific context. Never rely on context window alone for production agents.

MCP: MODEL CONTEXT PROTOCOL

WHAT IS MCP?

- Standardised protocol for exposing tools to AI agents
- Tool registration with JSON schemas
- Transport: stdio (local) or HTTP/SSE (remote)
- Any agent from any framework can use your MCP server

WHY MCP?

- Without MCP: every agent framework has its own tool format
- With MCP: one server, any client
- Like REST APIs for humans, MCP is APIs for AI agents

```
from kailash_mcp import MCPServer, tool
from kailash_ml import DataExplorer

server = MCPServer("ml-tools")

@server.tool
def profile_dataset(
    path: str,
) → dict:
    """Profile a dataset and return
    summary statistics."""
    explorer = DataExplorer()
    report = explorer.profile(path)
    return report.to_dict()

@server.tool
def train_model(
    dataset: str, target: str,
) → dict:
    """Train a classifier on the
    given dataset."""
    # ... implementation ...

server.run(transport="stdio")
```

A2A: AGENT-TO-AGENT COMMUNICATION

STRUCTURED COMMUNICATION

- Agents exchange typed messages, not free-form text
- Agent Cards: metadata about capabilities, tools, supported tasks
- Task lifecycle: submit → working → input-required → completed
- Streaming support for long-running tasks

SECURITY CONSIDERATIONS

- **Data leakage:** Agents must not share data beyond their authorisation
- **Prompt injection:** One agent's output becomes another's input — sanitise
- **Escalation attacks:** Agent A asks Agent B to do something Agent A is not allowed to do
- **Lesson 6.7:** PACT governance solves all three

EXERCISE 6.6: MULTI-AGENT ML PIPELINE

TASKS

1. Build multi-agent pipeline: DataScientist → FeatureEngineer → ModelSelector → ReportWriter
2. Build MCP server exposing ML tools
3. Test cross-agent coordination
4. Verify structured outputs between agents

ASSESSMENT CRITERIA

- Multi-agent pipeline produces correct end-to-end results
- MCP server functional and callable from agents
- Agents communicate via structured outputs (not free-form)
- Each agent stays within its specialisation

LESSON 6.7: AI GOVERNANCE ENGINEERING

LEARNING OBJECTIVES

- Implement PACT governance with D/T/R addressing
- Define and enforce operating envelopes for agents
- Implement budget cascading across agent hierarchies
- Test governance rules (verify denied access stays denied)
- Create audit trails for compliance

Design principle: This is ENGINEERING. Students implement access controls, test them, and verify they work. No philosophical discussion of AI ethics. The code IS the governance.

PACT: D/T/R ADDRESSING

DEPARTMENT / TEAM / ROLE

- **Department (D):** The top-level organisational unit
- **Team (T):** A team nested inside a department
- **Role (R):** The responsible role that owns a D or T leaf

Grammar: Dash-delimited path. Every D or T MUST be immediately followed by exactly one R.
Example: D1-R1-T1-R1 (department 1 head owns team 1, delegated to its responsible).

GOVERNANCEENGINE

```
from pact import (
    GovernanceEngine, load_org_yaml, Address)

loaded = load_org_yaml("finance_org.yaml")
engine = GovernanceEngine(loaded.org_definition)

# Role address grammar: D1-R1-T1-R1
# D1    = department 1
# R1    = that department's head
# T1-R1 = team 1, owned by its responsible
addr = Address.parse("D1-R1-T1-R1")
verdict = engine.verify_action(
    role_address="D1-R1-T1-R1",
    action="read_pii",
    context={
        "data_classification": "confidential"},
)
# verdict.allowed, verdict.reason,
# verdict.level ∈ {allowed, blocked,
# warn, audit}
```

OPERATING ENVELOPES

WHAT ARE OPERATING ENVELOPES?

- A `ConstraintEnvelopeConfig` defines the boundaries of what a role can do
- **Five canonical dimensions:** Financial, Operational, Temporal, Data Access, Communication — plus a confidentiality clearance and a delegation-depth cap
- **Monotonic tightening:** child envelopes must be $\leq$ parent on every dimension; the framework catches violations via `RoleEnvelope.validate_tightening`

Fail-closed: If the governance check fails (error, timeout), access is **denied**. Never fail-open. A broken guard is not a free pass.

```
from pact import (
    ConstraintEnvelopeConfig,
    FinancialConstraintConfig,
    OperationalConstraintConfig,
    TemporalConstraintConfig,
    DataAccessConstraintConfig,
    CommunicationConstraintConfig,
    ConfidentialityLevel,
    RoleEnvelope,
    MonotonicTighteningError,
)

parent = ConstraintEnvelopeConfig(
    id="parent",
    description="Analyst envelope",
    confidentiality_clearance=
        ConfidentialityLevel.RESTRICTED,
    financial=FinancialConstraintConfig(
        max_spend_usd=5.00),
    operational=OperationalConstraintConfig(
        allowed_actions=[
            "data_analysis",
            "model_training"]),
    temporal=TemporalConstraintConfig(),
    data_access=DataAccessConstraintConfig(),
    communication=
        CommunicationConstraintConfig(),
    max_delegation_depth=3,
)

# Monotonic tightening enforced by the
# framework - child must be ≤ parent
# on every dimension.
try:
    RoleEnvelope.validate_tightening(
        parent, child)
except MonotonicTighteningError as exc:
    print(f"Escalation blocked: {exc}")
```

BUDGET CASCADING

HOW IT WORKS

- Parent agent has a total budget: \$10.00
- Parent allocates portions to child agents
- Child agents cannot exceed their allocation
- When a child's budget runs out, it stops and reports partial results
- Parent can reallocate remaining budget dynamically

```
from pact import (
    ConstraintEnvelopeConfig,
    FinancialConstraintConfig,
    OperationalConstraintConfig,
    TemporalConstraintConfig,
    DataAccessConstraintConfig,
    CommunicationConstraintConfig,
    ConfidentialityLevel,
)

# Budget cascading is expressed as financial
# dimensions on each role's envelope. The
# parent envelope has max_spend_usd=10.00;
# each child's cap must be ≤ 10.00, and the
# monotonic-tightening check enforces it.
supervisor_env = ConstraintEnvelopeConfig(
    id="supervisor_env",
    description="Supervisor - $10 total",
    confidentiality_clearance=
        ConfidentialityLevel.CONFIDENTIAL,
    financial=FinancialConstraintConfig(
        max_spend_usd=10.00),
    operational=OperationalConstraintConfig(
        allowed_actions=[
            "analyse", "delegate"]),
    temporal=TemporalConstraintConfig(),
    data_access=DataAccessConstraintConfig(),
    communication=
        CommunicationConstraintConfig(),
    max_delegation_depth=3,
)

# Child envelopes follow the same shape with
# max_spend_usd ∈ (1.00, 2.00, 4.00, 1.00).
# The framework halts a child when its cap
# is exhausted; the supervisor's remaining
# budget is never touched.
```

GOVERNEDSUPERVISOR: GOVERNANCE BUILT IN

AGENT + GOVERNANCE = GOVERNEDSUPERVISOR

```
from part import GovernanceEngine, load_org_yaml
from kaizen_agents import GovernedSupervisor

loaded = load_org_yaml({"finance_org.yaml"})
engine = GovernanceEngine(loaded.org_definition)

governed = GovernedSupervisor(
    model="gpt-4o-mini",
    budget_upper=100,
    tools=["data_analysis"],
    data_clearance="restricted",
)

async def executor(spec, inputs):
    # Real LLM call or offline stub.
    return {"result": "...", "cost": 0.01,
            "prompt_tokens": 100,
            "completion_tokens": 50}

# Governed run - supervisor plans, executor runs.
result = await governed.run(
    objective="Analyze Q4 revenue data",
    executor_node=executor,
)
# result.success, result.budget_consumed, result.audit_trail

# Asking for an action outside the envelope:
verdict = engine.verify_action(
    role_address="01-PI-11-R1",
    action="delete_all_records",
    content={"data_classification": "restricted"},
)
assert not verdict.allowed # blocked + reason in verdict.reason
```


GOVERNANCE TESTING: PROVING SAFETY

THE TESTING IMPERATIVE

- Governance without tests is governance theatre
- Test that allowed actions succeed
- Test that denied actions are blocked
- Test that envelopes tighten correctly
- Test that budget enforcement works at zero

```
import pytest
from pact import (
    RoleEnvelope, MonotonicTighteningError)

def test_analyst_can_read_data():
    verdict = engine.verify_action(
        role_address="D1-R1-T1-R1",
        action="read_data",
        context={"data_classification":
            "restricted"})
    assert verdict.allowed

def test_analyst_cannot_delete():
    verdict = engine.verify_action(
        role_address="D1-R1-T1-R1",
        action="delete_customer_data",
        context={"data_classification":
            "restricted"})
    assert not verdict.allowed

def test_envelope_tightening():
    # Loosening raises structurally:
    with pytest.raises(
        MonotonicTighteningError):
        RoleEnvelope.validate_tightening(
            parent_env, loosened_child_env)

def test_zero_budget_stops_spent():
    result = await governed_run(
        objective="Analyse data",
        executor=mock_executor)
    assert result.budget_consumed <= 2.00
```

Red team your own governance. If you cannot write a test that proves your access control works, it does not work.

AUDIT TRAILS & CLEARANCE LEVELS

AUDIT TRAIL

- Every access decision is logged
- Who requested, what was requested, what was the decision, why
- Immutable: append-only log
- Required for: regulatory compliance, incident investigation, accountability

```
# Every governed supervisor keeps a
# hash-chained audit trail:
trail = governed.audit.to_list()
for entry in trail[-10:]:
    print(f"{entry['timestamp']}: "
          f"{entry['action']} → "
          f"{entry['verdict']}: "
          f"{entry['reason']}")
# Tamper-evidence check:
assert governed.audit.verify_chain()
```

CLEARANCE LEVELS

Level	Access
Public	Public data, read-only tools
Internal*	Internal data, analysis tools
Confidential	PII, financial data
Restricted	System admin, model deployment

Clearance levels map to role addresses. A role at D1-R1-T1-R1 might carry "Confidential" clearance; a public support bot carries "Public".

*In pact 0.8.1 the canonical five-level ladder is PUBLIC < RESTRICTED < CONFIDENTIAL < SECRET < TOP_SECRET. The course's "Internal" label is a historical alias that maps to RESTRICTED at the same tier as "Restricted" in the course ladder.

EXERCISE 6.7: GOVERNED MULTI-AGENT SYSTEM

TASKS

1. Define D/T/R organisational structure
2. Set operating envelopes (task and role)
3. Implement budget cascading across agent hierarchy
4. Write governance tests (allow + deny)
5. Generate and inspect audit trail

ASSESSMENT CRITERIA

- Governance correctly denies unauthorised access
- Operating envelopes enforce boundaries
- Budget cascading works (agents stop at limit)
- Tests prove governance works
- Audit trail captures all decisions

LESSON 6.8: CAPSTONE — FULL PRODUCTION PLATFORM

LEARNING OBJECTIVES

- Deploy a complete AI system with Nexus (API + CLI + MCP)
- Implement authentication and authorisation
- Integrate all Kailash packages into a production pipeline
- Debug agent reasoning chains
- Monitor deployed models for drift

Prerequisites: All of M6 (6.1–6.7). Scaffolding: ~40%. Students connect existing components.

NEXUS: ONE CODEBASE, THREE INTERFACES

NEXUS MULTI-CHANNEL DEPLOYMENT

```
from nexus import Nexus
from kailash.workflow.builder import WorkflowBuilder

# Describe the analysis workload as a built workflow
analysis_wf = WorkflowBuilder()
analysis_wf.add_node(
    "PythonCodeNode",
    "analyse",
    {"code": "result = governed_agent.run_sync(parameters['query'])"},
)

# Nexus exposes it as API + CLI + MCP simultaneously
app = Nexus() # JWT middleware + rate limiting configured in project settings
app.register("analysis", analysis_wf.build())
app.start() # serve on the configured port
```

One Codebase → REST API + CLI + MCP Server

AUTHENTICATION & AUTHORISATION

RBAC (ROLE-BASED ACCESS CONTROL)

- Users have roles (admin, analyst, viewer)
- Roles have permissions (read, write, deploy)
- Permissions map to Nexus endpoints
- PACT D/T/R addresses map to RBAC roles

```
# Nexus oem authentication (JWT): PACT
# oem authentication (verify_action).
from nexus import Nexus

app = Nexus() # JWT middleware applied
app.register("analyse", analyse_wf.build())
app.register("deploy", deploy_wf.build())

# Inside each node (pseudo-code):
# caller_role = parameters["auth"]["role_address"]
# verdict = engine.verify_action(
#     role_address=caller_role,
#     action="deploy",
#     context={"data_classification":
#             "restricted"})
#
# If not verdict.allowed:
#     raise PermissionError(verdict.reason)
#
```

JWT AUTHENTICATION

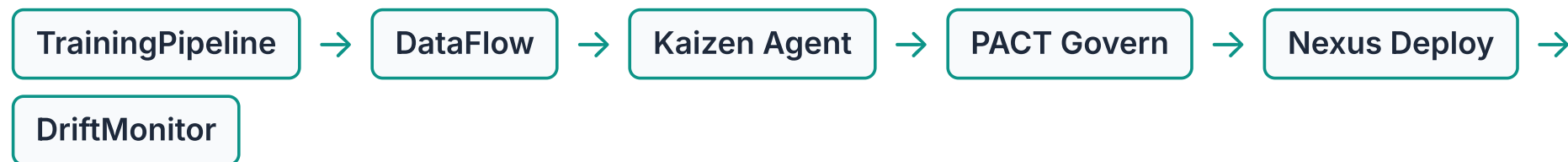
```
# Client-side
import requests

token = requests.post(
    "http://localhost:8000/auth/token",
    json={"username": "analyst1",
          "password": "xxxx"}
).json()["token"]

# Authenticated request
response = requests.post(
    "http://localhost:8000/api/analyse",
    headers={
        "Authorization": f"Bearer {token}"
    },
    json={"query": "Analyse Q4 data"}
)
```

Unauthenticated requests must be rejected with 401.
No exceptions.

FULL PLATFORM INTEGRATION



THE FULL STACK

Step	Package	Purpose
1	kailash-ml	Train model
2	kailash-dataflow	Persist to database
3	kailash-kaizen	Wrap in agent
4	kailash-pact	Add governance
5	kailash-nexus	Deploy multi-channel
6	kailash-ml	Monitor for drift

This is the entire MLFP stack. Six packages, one pipeline. Every module you have completed contributes a piece. M6 is where it all comes together.

Debugging traces: Every agent action, governance decision, and API call is traceable. When something fails in production, you can reconstruct the entire chain.

PRODUCTION MONITORING: DRIFTMONITOR

WHY MONITOR?

- Models degrade over time as data distributions shift
- An accurate model today may be wrong tomorrow
- Drift detection triggers retraining or alerts

TYPES OF DRIFT

- **Data drift:** Input distribution changes
- **Concept drift:** Relationship between inputs and outputs changes
- **Performance drift:** Model accuracy degrades

```
from kailash_ml import DriftMonitor

monitor = DriftMonitor(
    reference_data=training_data,
    model_id="sentiment-v1",
    alert_threshold=0.05,
)

# In production: check each batch
for batch in production_batches:
    report = monitor.check(batch)
    if report.drift_detected:
        print(f"DRIFT: {report.feature} "
              f"(p={report.p_value:.4f})")
        # Trigger retraining pipeline
        retrain(model_id="sentiment-v1")
```


DEBUGGING AGENT REASONING CHAINS

THE TRACE

```
result = agent.run(
    "Analyse HDB resale data",
    trace=True
)

for step in result.trace:
    print(f"[{step.type}] {step.content}")
    if step.type == "action":
        print(f"  Tool: {step.tool}")
        print(f"  Args: {step.args}")
        print(f"  Cost: ${step.cost:.4f}")
    if step.type == "governance":
        print(f"  Decision: {step.decision}")
        print(f"  Reason: {step.reason}")
```

COMMON DEBUGGING PATTERNS

Symptom	Likely Cause
Agent loops forever	Ambiguous goal or missing tool
Wrong tool selected	Tool descriptions too similar
Governance blocked	Missing permission in envelope
Budget exhausted	Too many retries; simplify task
Incoherent reasoning	Context window overflow

TESTING AGENTIC SYSTEMS

WHAT TO TEST

- **Tool correctness:** Each tool produces expected output for known input
- **Reasoning quality:** Agent selects correct tools for a given task
- **Governance:** Blocked actions stay blocked
- **Budget:** Agent stops at budget limit
- **End-to-end:** Full task produces correct final result

```
def test_tool_selection():  
    """Agent selects explore tool for  
    data analysis tasks."""  
    result = agent.run(  
        "What is the average price?",  
        trace=True)  
    tools_used = [s.tool  
        for s in result.trace  
        if s.type == "action"]  
    assert "explore_data" in tools_used  
  
def test_end_to_end():  
    """Full analysis produces a report."""  
    result = agent.run(  
        "Analyse and report on Q4 data")  
    assert result.success  
    assert "average" in result.output  
    assert result.cost < 1.00
```

INFERENCE OPTIMISATION: PRODUCTION SERVING

VLLM: PRODUCTION LLM SERVING

- PagedAttention: efficient KV-cache memory management
- Continuous batching: maximise GPU throughput
- Tensor parallelism: split model across GPUs
- OpenAI-compatible API: drop-in replacement

```
pip install vllm
python -m vllm.entrypoints.openai.api_server \
  --model meta-llama/Llama-3-8B \
  --tensor-parallel-size 2
```

FLASH ATTENTION

- Tiling-based attention computation
- Reduces memory from $O(n^2)$ to $O(n)$
- 2-4x faster than standard attention
- Used by default in modern serving frameworks

You do not implement these. You configure them. The engineering skill is knowing which optimisation applies to your deployment constraint (latency, throughput, memory).

MULTIMODAL LLMS (BRIEF AWARENESS)

VISION-LANGUAGE MODELS

- **GPT-4V:** Text + image understanding
- **Gemini:** Native multimodal (text, image, video, audio)
- **LLaVA:** Open-source vision-language model

APPLICATIONS

- Document understanding (OCR + reasoning)
- Chart interpretation (read graphs, extract data)
- Visual question answering
- Multimodal RAG: retrieve images + text

Trajectory: LLMs are becoming multimodal by default. The text-only era is ending. Future agents will see, hear, and read simultaneously. The same Kaizen patterns (Delegate, BaseAgent, tools) apply to multimodal models.

For this module: Awareness only. The engineering patterns you have learned (prompting, agents, governance) transfer directly to multimodal systems. The tools change; the architecture does not.

EXERCISE 6.8: CAPSTONE — DEPLOY A GOVERNED AI PLATFORM

TASKS

1. Deploy the M6 multi-agent system via Nexus
2. Add JWT authentication with RBAC
3. Integrate DriftMonitor for production monitoring
4. Test end-to-end: query via API, CLI, and MCP
5. Verify governance enforces access controls at deployment level

ASSESSMENT CRITERIA

- System deployed and accessible via all 3 channels
- Auth works (unauthenticated requests rejected with 401)
- Drift monitoring active and alerting
- Governance enforced in production
- Complete audit trail generated

KEY FORMULA RECAP: ATTENTION MECHANISM

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{d_k}\right) V$$

From M5: the foundation of every LLM. Q = queries, K = keys, V = values, d_k = key dimension

WHERE IT APPEARS IN M6

- **6.1:** LLMs use multi-head attention at every layer
- **6.2:** LoRA targets attention weight matrices (q_proj, v_proj)
- **6.4:** Dense retrieval uses attention-based embeddings
- **6.8:** Flash Attention optimises this computation in production

The d_k scaling prevents dot products from growing too large, which would push softmax into saturated regions (near-zero gradients). This is why training is stable at scale.

M6 FORMULA SUMMARY

$$W = W_0 + BA \quad (\text{LoRA})$$

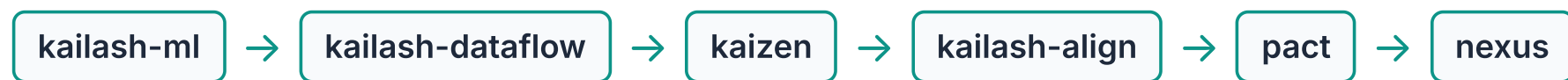
$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

$$\hat{A}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})} \quad (\text{GRPO advantage})$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{d_k} \right) V$$

Four formulas, four concepts: LoRA decomposes weight updates, DPO aligns with preferences, GRPO normalises rewards within a group, Attention is the computation substrate for all of them.

THE COMPLETE KAILASH STACK



Package	M6 Role	Key APIs Used
kaizen / kaizen_agents	LLM calls, agent framework	BaseAgent, Signature, Delegate, ReActAgent, GovernedSupervisor
kailash-align	Fine-tuning, alignment	AlignmentPipeline, AlignmentConfig, AdapterRegistry
pact	Governance, access control	GovernanceEngine, load_org_yaml, ConstraintEnvelopeConfig, RoleEnvelope
nexus	Multi-channel deployment	Nexus, app.register(name, built_workflow), JWT middleware
kailash-ml	Training, monitoring	TrainingPipeline, DriftMonitor, DataExplorer
kailash-mcp	Tool protocol server	MCPServer, @tool decorator

THE M6 JOURNEY



THE ARC

- **Understand** (6.1): How LLMs work
- **Customise** (6.2-6.3): Fine-tune and align
- **Ground** (6.4): Connect to real data
- **Empower** (6.5-6.6): Give agency and coordination
- **Govern** (6.7): Enforce safety
- **Ship** (6.8): Deploy to production

Every concept produces runnable code. There is no theory-only lesson in M6. If you completed the exercises, you have built a production-ready AI platform.

WHAT MAKES THIS MODULE DIFFERENT

TYPICAL LLM COURSE

"Call the API, parse the response, build a chatbot."

MLFP MODULE 6

Implement LoRA from scratch. Derive DPO. Build governed multi-agent systems. Deploy with monitoring.

TYPICAL AGENT TUTORIAL

"Install LangChain, run the example, done."

MLFP MODULE 6

Design agents with the hiring framework. Enforce cost budgets. Test governance. Ship production.

The difference: We teach you to build systems that are *safe, tested, governed, and deployed*. Not just systems that work in a notebook.

DECISION FRAMEWORK: WHEN TO USE WHAT

Problem	Solution	Lesson
Need LLM output in production	Structured prompting + Kaizen Signature	6.1
Model needs domain knowledge	LoRA fine-tuning via AlignmentPipeline	6.2
Model output quality is poor	DPO alignment with preference data	6.3
Need factual, grounded answers	RAG with hybrid retrieval	6.4
Task requires multi-step reasoning	ReAct agent with tools	6.5
Complex task with sub-problems	Multi-agent orchestration	6.6
Need access control and safety	PACT governance + operating envelopes	6.7
Ship to users	Nexus multi-channel deployment	6.8

COMMON MISTAKES TO AVOID

Mistake	Why It Fails	Correct Approach
Fine-tuning when RAG would work	Expensive, slow, model goes stale	Try RAG first; fine-tune only if insufficient
Agent without cost budget	Runaway API spending	Always set max_cost on every agent
No governance on agents	Security liability	GovernedSupervisor with pact operating envelopes
Testing LLM exact output	Non-deterministic; tests flake	Test tool selection, governance, budget
Deploying without monitoring	Model degrades silently	DriftMonitor on every deployed model
Using agents for simple tasks	Adds cost and latency	Pipeline first; agent only if reasoning needed

M6 IN THE MLFP CURRICULUM

WHAT M6 BUILDS ON

- **M1:** Python, Polars, data pipelines
- **M2:** Statistics, probability (evaluation foundations)
- **M3:** Supervised ML, pipelines (TrainingPipeline)
- **M4:** NLP, embeddings (retrieval foundations)
- **M5:** Deep learning, transformers, RL (architecture + PPO)

WHAT M6 DELIVERS

- Complete LLM engineering skillset
- From-scratch implementations of key algorithms
- Production deployment with governance
- The full Kailash stack integrated end-to-end

M6 is the culmination of the MLFP programme. Every module contributed a building block. M6 assembles them into a production AI platform.

THE ML SPECTRUM: WHERE LLMS FIT

Module	Spectrum Position
M1-M2	Data understanding & statistics
M3	Supervised prediction
M4	Pattern discovery & language
M5	Representation learning
M6	Reasoning, agency, governance

M6 LESSON SPECTRUM

- **6.1:** Semantic feature learning at scale
- **6.2:** Customising learned representations
- **6.3:** Engineering the training signal
- **6.4:** Knowledge-augmented generation
- **6.5-6.6:** Autonomous ML systems
- **6.7:** Governed AI systems
- **6.8:** Production deployment

SELF-ASSESSMENT: CAN YOU...

FOUNDATIONS (EVERYONE)

- Explain how LLMs are pre-trained?
- Apply 5+ prompting techniques?
- Use Kaizen Delegate for structured output?
- Build a RAG pipeline?
- Create a ReAct agent with tools?
- Set cost budgets on agents?
- Implement PACT access controls?
- Deploy via Nexus?

THEORY (STRETCH)

- Implement LoRA from scratch?
- Derive and implement the DPO loss?
- Explain GRPO vs DPO tradeoffs?
- Evaluate RAG with RAGAS metrics?
- Build an MCP server?
- Write governance tests?

ADVANCED (EXPERT)

- Explain model merging (TIES, DARE)?
- Discuss QLoRA and quantisation?
- Compare vLLM inference optimisations?

END-OF-MODULE ASSESSMENT

CAPSTONE PROJECT

- Deploy a complete governed AI system
- Must include: prompting, fine-tuning, RAG, agents, governance, deployment
- Presentation: 15 minutes + 5 minutes Q&A
- Demo: live queries via API, CLI, and MCP

COMPREHENSIVE QUIZ

- Covers all 8 lessons
- Mix of: conceptual, code interpretation, formula application
- Open-book: you may reference your exercises
- Duration: 60 minutes

Assessment focus: Can you build it, deploy it, govern it, and explain why your design decisions are correct? The code IS the answer.

RESOURCES & REFERENCES

PAPERS

- Hu et al. (2021) — LoRA: Low-Rank Adaptation
- Houlsby et al. (2019) — Parameter-Efficient Transfer Learning
- Rafailov et al. (2023) — DPO: Direct Preference Optimization
- Shao et al. (2024) — DeepSeek-R1 (GRPO)
- Yao et al. (2023) — ReAct: Reasoning and Acting
- Wei et al. (2022) — Chain-of-Thought Prompting
- Kojima et al. (2022) — Zero-shot CoT
- Lewis et al. (2020) — RAG: Retrieval-Augmented Generation

KAILASH DOCUMENTATION

- kailash-kaizen: Agent framework & Delegate
- kailash-align: Fine-tuning & alignment pipelines
- kailash-pact: Governance & access control
- kailash-nexus: Multi-channel deployment
- kailash-mcp: MCP server framework

TOOLS

- lm-eval-harness: Evaluation framework
- RAGAS: RAG evaluation
- vLLM: Production LLM serving

KEY TAKEAWAYS

1. **Prompting is an engineering skill.** Start simple, escalate by measured need.
2. **LoRA democratised fine-tuning.** 256x fewer parameters, same quality.
3. **DPO eliminated the reward model.** Alignment is now accessible.
4. **RAG is the production default.** Ground LLMs in facts, not memory.
5. **Agents need tools, not just prompts.** ReAct = reasoning + action.
6. **Multi-agent systems need coordination protocols.** MCP for tools, A2A for agents.
7. **Governance is engineering, not philosophy.** Code it. Test it. Ship it.
8. **Deploy with monitoring.** A model without DriftMonitor is a liability.

FOR THE INSTRUCTOR: PACING NOTES

Lesson	Estimated Time	Key Risk
6.1 LLM Fundamentals	2.5 hours	Prompt engineering feels "too easy" — push to structured output
6.2 Fine-tuning	3.5 hours	From-scratch implementations need careful debugging
6.3 DPO/GRPO	2.5 hours	DPO formula derivation is dense — go slow
6.4 RAG	3 hours	Chunking mistakes cascade — verify before retrieval
6.5 Agents	3 hours	Cost overruns if budgets not enforced
6.6 Multi-Agent	2.5 hours	Agent coordination debugging is non-trivial
6.7 Governance	2.5 hours	Students may skip tests — make tests mandatory
6.8 Capstone	4 hours	Integration issues — allocate debugging time

DISCUSSION QUESTIONS

REFLECT

1. When should you use RAG vs fine-tuning?
Give a concrete example where each is the right choice.
2. What happens if you deploy an agent without governance? What is the worst case?
3. Why does DPO not need a reward model while RLHF does? Explain in plain language.

APPLY

4. Your company has 10,000 internal documents. Describe how you would build a system that answers employee questions about company policy.
5. You need to deploy an AI assistant that handles customer enquiries. What Kailash packages would you use and why?

MODULE 6 COMPLETE

MACHINE LEARNING WITH LANGUAGE MODELS AND AGENTIC WORKFLOWS

You can now build, fine-tune, align, ground, govern, and deploy production AI systems.

"The goal is not to use AI. The goal is to deploy AI that is safe, tested, governed, and useful."

MLFP — ML Foundations for Professionals — Terrene Open Academy

Powered by Kailash Python SDK — Terrene Foundation